A Project Report

On

ChatSphere

(Real-Time Chat Application)

Submitted in partial fulfillment of the requirement for the award of degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

FULL STACK – II (24CSP – 339)

Submitted by

Prionti Das (24BAI71005)

Submitted to

Ms. Lovleen Kaur (E20454)

APEX INSTITUTE OF TECHNOLOGY, CSE – AIML

CHANDIGARH UNIVERSITY, GHARUAN

2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported and guided me throughout the successful completion of this project.

First, I extend my heartfelt thanks to my project mentor, **Ms. Lovleen Kaur**, for her valuable guidance, continuous support, encouragement and constructive feedback. I am also thankful to the faculty members of Apex Institute of Technology (CSE-AIML), Chandigarh University, for providing the necessary knowledge and academic support.

I would also like to thank my friends and classmates for their cooperation and suggestions. Finally, I am deeply grateful to my family for their constant motivation and encouragement, which helped me complete this project successfully.

Thank you to everyone who contributed directly or indirectly to the completion of this project.

Prionti Das

24BAI71005

CERTIFICATE

This is to certify that the project report entitled **"ChatSphere – Real-Time Chat Application"** is the original work carried out by **Prionti Das, UID: 24BAI71005**, under my supervision, in partial fulfilment of the requirements for the degree of **Bachelor of Engineering in Computer Science and Engineering (AIT – AIML)** at Chandigarh University.

The work presented in this report is based on the research, design, implementation, and findings carried out by the student in accordance with the academic norms and guidelines of Chandigarh University. The report has been reviewed and found to meet the prescribed academic standards and project requirements.

SIGNATURE

Ms. Lovleen Kaur (E20454)

Assistant Professor (APEX – CSE)

Date: 29/10/2026

Table of Content

Chapter	Content	Page No.
	List of Tables	I
	List of Figures	II
	Abstract	III
1.	**Introduction**	1-2
	1.1 Background	1
	1.2 Problem Statement	1
	1.3 Proposed System	2
	1.4 Scope of the Project	2
2.	**Objectives**	3
3.	**Software Requirements**	4
	3.1 Software Requirements	4
4.	**Theory**	5-14
	4.1 Full-Stack Web Development	5
	4.2 Client-Server Architecture	5-6
	4.3 Java Programming Language	6-7
	4.4 Spring Boot Framework	7-8

	4.5 Frontend Technologies	8-10
	4.6 HTTP and Client-Server Communication	10-11
	4.7 User Registration	11
	4.8 Theory of Real-Time Chat Applications	12
	4.9 Security in Web Applications	13
	4.10 Error Handling and Input Validation	13-14
	4.11 Maintainability and Future Enhancement	14
5.	**Procedure**	15-18
	5.1 Project Setup and Configuration	15
	5.2 Backend Development	15
	5.3 Frontend Development	15-16
	5.4 User Registration and Chat Interface	16
	5.5 Frontend and Backend Integration	16
	5.6 Execution and Testing	17-18
6.	**Code**	19-23
	6.1 Main Application Class	19
	6.2 Application Configuration	19
	6.3 User Model	19

	6.4 Registration Controller	20
	6.5 Chat Message Model	20
	6.6 WebSocket Configuration	20-21
	6.7 Chat WebSocket Handler	21
	6.8 Main Webpage	21-22
	6.9 CSS Styling	22
	6.10 Frontend JavaScript	22-23
7.	**Results**	24-32
	7.1 Sign-In Interface	24
	7.2 Main Chat Dashboard	25
	7.3 Conversation from the First User's View	26
	7.4 Conversation from the Second User's View	27
	7.5 Ideas Chat Room	28
	7.6 Friend Conversation Interface	28-29
	7.7 General Community Chat	29
	7.8 Message Exchange Across Two Browser Windows	30
	7.9 Message Interaction Options	30-31

	7.10 Overall Results	31-32
8.	**Conclusion and Future Scope**	33-34
	8.1 Conclusion	33
	8.2 Future Scope	34

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

List of Tables

Table No.	Description	Page No.
Table 1	Software Requirements of ChatSphere	4
Table 2	Technologies Used in ChatSphere	8-9
Table 3	Functional Testing of ChatSphere	17-18
Table 4	Summary of ChatSphere Features	31

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

List of Figure

Figure No.	Description	Page No.
Fig. 7.1	ChatSphere Sign-In Interface	24
Fig. 7.2	ChatSphere Main Chat Dashboard	25
Fig. 7.3	Chat Conversation from the First User's View	26
Fig. 7.4	Chat Conversation from the Second User's View	27
Fig. 7.5	Ideas Chat Room Interface	28
Fig. 7.6	Friend Conversation Interface	28
Fig. 7.7	General Community Chat Interface	29
Fig. 7.8	Message Exchange Across Two Browser Windows	30
Fig. 7.9	Message Interaction Options	30

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Abstract

ChatSphere is a web-based Real-Time Chat Application developed to provide a simple and user-friendly platform for online communication. It allows users to register and access a chat interface designed for exchanging messages. The application aims to make digital communication convenient and accessible.

The project uses **Java and Spring Boot** for backend development and **HTML, CSS, and JavaScript** for the frontend. It demonstrates the integration of frontend and backend technologies, server-side programming, and client-server communication in full-stack development.

In the future, ChatSphere can be enhanced with features such as group chats, message history, notifications, and improved security. Overall, the project provides practical knowledge of modern web application development and serves as a foundation for building a communication platform.

CHAPTER 1

INTRODUCTION

ChatSphere is a web-based Real-Time Chat Application developed to provide a convenient platform for online communication between users. In today's digital world, messaging applications play an important role in connecting people and enabling quick information exchange. ChatSphere aims to provide a simple and interactive interface that makes online communication easier and more accessible.

The project is developed using Java and Spring Boot for backend operations, along with HTML, CSS, and JavaScript for the frontend. It includes a user registration process and a chat interface designed to support message exchange. The project demonstrates the practical implementation of full-stack development concepts by integrating frontend and backend components into a web application.

1.1 Background

Online communication has become an essential part of everyday life, education, and professional activities. Chat applications allow users to communicate conveniently without being physically present in the same location. ChatSphere is developed to understand and implement the basic concepts involved in building a web-based messaging application using modern web technologies.

1.2 Problem Statement

Traditional communication methods may not always provide the speed and convenience required for digital interaction. Therefore, there is a need for a simple web-based platform that provides users with an accessible interface for communication. ChatSphere addresses this need by providing a registration process and a dedicated chat interface as the foundation of a messaging system.

1.3 Proposed System

The proposed system, ChatSphere, uses Spring Boot to handle backend operations and web technologies to create an interactive frontend. Users can register and access the application's interface. The system is designed to provide a foundation for communication between users and can be further enhanced with additional messaging features.

1.4 Scope of the Project

The scope of ChatSphere includes developing a web-based application, implementing user registration, designing a chat interface, and integrating frontend and backend technologies. The project helps demonstrate the fundamentals of full-stack application development. In the future, features such as group messaging, message history, notifications, and enhanced security can be added to improve the application's functionality.

CHAPTER 2

OBJECTIVES

The main objective of ChatSphere is to develop a web-based chat application that provides a simple and user-friendly platform for online communication. The project focuses on integrating frontend and backend technologies to build an interactive messaging application.

The major objectives of the project are:

1. **User Registration:** To provide a registration system that allows users to register and access the application.
2. **User-Friendly Interface:** To design an attractive and interactive chat interface using HTML, CSS, and JavaScript.
3. **Backend Development:** To use Java and Spring Boot for handling server-side operations and application logic.
4. **Frontend and Backend Integration:** To integrate frontend and backend components for the proper functioning of the application.
5. **Practical Learning:** To understand full-stack development concepts, client-server communication, and the development of web-based applications.

CHAPTER 3

SOFTWARE REQUIREMENTS

S. No.	Software / Technology	Purpose
1	Windows OS	Development environment
2	Java	Backend programming
3	Spring Boot	Backend application framework
4	HTML	Webpage structure
5	CSS	Interface styling
6	JavaScript	Frontend interactivity
7	Visual Studio Code	Code editing and project management
8	Maven	Dependency management and application build
9	Google Chrome	Running and testing the web interface

Table 1: Software Requirements of ChatSphere

CHAPTER 4

THEORY

4.1 Full-Stack Web Development

Full-stack web development is the process of developing both the frontend and backend components of a web application. The frontend represents the part of the application that users can see and interact with, whereas the backend handles application logic, processes requests, and performs server-side operations.

A full-stack application requires proper communication between these components to provide the required functionality. The frontend collects user input and displays information, while the backend processes requests and returns appropriate responses.

ChatSphere is developed using full-stack web development concepts. The frontend is created using HTML, CSS, and JavaScript, while Java and Spring Boot are used for backend development. These technologies work together to provide a web-based environment for user registration and chat-related functionality.

Full-stack development offers several advantages, including better coordination between application components, easier maintenance, and flexibility for future improvements. It also helps developers understand how different technologies work together to create a complete web application.

4.2 Client-Server Architecture

Client-server architecture is a computing model in which a client requests a service and a server processes the request and provides a response. This architecture is widely used in websites, online applications, messaging platforms, and other network-based systems.

The client is generally the user's web browser, which provides the interface for interacting with the application. The server receives requests from the client, performs the required operations, and sends responses back to the browser.

In ChatSphere, the web browser acts as the client, while the Spring Boot application performs backend operations. When a user interacts with the application, the relevant request can be processed by the backend, depending on the functionality involved.

The main components of client-server architecture are:

1. **Client:** Provides the user interface and accepts user input.
2. **Server:** Processes incoming requests and executes application logic.
3. **Communication Protocol:** Defines how information is exchanged between the client and server.
4. **Response:** Contains the result or information returned by the server to the client.

This architecture separates user interface design from backend processing, making the application easier to develop, test, and maintain. Additional features can also be introduced without redesigning every part of the system.

4.3 Java Programming Language

Java is a high-level, object-oriented programming language widely used for developing web applications, desktop software, enterprise systems, and backend services. It supports platform-independent application development through the Java Virtual Machine (JVM).

Java follows object-oriented programming principles such as encapsulation, inheritance, abstraction, and polymorphism. These principles help organize application code into reusable and manageable components.

In ChatSphere, Java is used to implement backend logic through the Spring Boot framework. It provides the foundation for processing application requests and managing the operations required by the system.

Some important features of Java are:

- **Platform Independence:** Java applications can run on compatible systems that provide the required Java runtime.
- **Object-Oriented Structure:** Code can be organized into classes and objects.

- **Security Features:** Java provides runtime checks and language features that help build secure applications when used correctly.
- **Exception Handling:** Exceptions can be managed to handle unexpected situations during execution.
- **Rich Libraries:** Java provides libraries that reduce the need to develop common functionality from the beginning.

Java is suitable for backend development because it provides a structured programming environment and supports the development of maintainable applications.

4.4 Spring Boot Framework

Spring Boot is a framework built on the Spring ecosystem that simplifies the development and configuration of Java applications. It provides tools and conventions that help developers create applications with less manual configuration.

Traditional Spring applications may require considerable configuration before they can run. Spring Boot simplifies this process through features such as auto-configuration, starter dependencies, and embedded web servers.

ChatSphere uses Spring Boot as its backend framework. It provides the environment required to run the application and process supported server-side operations. It also makes it possible to organize backend code into components that perform specific responsibilities.

Important features of Spring Boot include:

1. **Auto-Configuration:** Configures many application components according to the libraries and settings present in the project.
2. **Starter Dependencies:** Simplify dependency management by grouping commonly required libraries.
3. **Embedded Server Support:** Allows web applications to run with an embedded server, reducing the need for a separately configured application server.
4. **Application Structure:** Supports organizing functionality into controllers, services, and other components where appropriate.

5. **Development Support:** Provides tools and conventions that simplify application execution and maintenance.

Spring Boot helps provide a structured backend for ChatSphere. Additional backend capabilities can be developed as the project grows, depending on the requirements of the application.

4.5 Frontend Technologies

The frontend is the visual and interactive part of a web application. It allows users to access application features, enter information, and view responses. A well-designed frontend improves usability by making navigation and interaction easier.

ChatSphere uses HTML, CSS, and JavaScript to develop its user interface.

Technology	Main Function	Application in ChatSphere
HTML	Defines webpage structure	Sign-in and chat interfaces
CSS	Controls appearance and layout	Colours, spacing, and interface design
JavaScript	Handles client-side interactions	User actions and interface updates
Java	Implements backend logic	Server-side application operations
Spring Boot	Supports backend development	Runs the web application and handles requests

HTTP	Supports request-response communication	Browser and backend communication where applicable

Table 2: Technologies Used in ChatSphere

4.5.1 HTML

HTML stands for HyperText Markup Language. It defines the structure and content of web pages using elements such as headings, paragraphs, forms, buttons, input fields, and containers.

In ChatSphere, HTML provides the basic structure of the application pages. It can be used to organize the registration form, input fields, chat area, and other interface elements.

Semantic and well-structured HTML helps make a web page easier to maintain and improves accessibility when appropriate elements and attributes are used.

4.5.2 CSS

CSS stands for Cascading Style Sheets. It is used to control the appearance and layout of web pages. CSS defines properties such as colours, fonts, spacing, borders, alignment, and element positioning.

In ChatSphere, CSS can be used to style registration forms, buttons, message areas, navigation components, and other interface elements. Consistent styling helps maintain a uniform appearance throughout the application.

CSS also supports responsive design through flexible layouts and media queries. Responsive design helps an interface adapt to different screen sizes, such as desktop monitors, tablets, and mobile devices.

4.5.3 JavaScript

JavaScript is a programming language used to add behaviour and interactivity to web pages. It can respond to user actions, validate input, modify webpage content, and communicate with backend services.

In ChatSphere, JavaScript supports frontend interactions and can manage user actions related to the application's interface. It can also be used to send requests to the backend and update the displayed information according to server responses.

When integrated properly with HTML and CSS, JavaScript helps create a more interactive user experience than a static webpage alone.

4.6 HTTP and Client-Server Communication

HTTP stands for Hypertext Transfer Protocol. It is a communication protocol used to transfer information between clients and web servers. A browser commonly uses HTTP to request web pages and communicate with backend endpoints.

HTTPS is the secure version of HTTP that uses encryption through TLS to protect communication in transit.

In a typical web application, communication follows a request-response pattern. The client sends a request to the server, the server processes it, and the server returns a response containing a status and, where appropriate, data.

Common HTTP methods include:

- **GET:** Requests information from a server.
- **POST:** Sends data to a server, commonly to create a resource or submit information.
- **PUT:** Replaces or updates a resource.
- **PATCH:** Applies a partial update to a resource.
- **DELETE:** Requests the deletion of a resource.

In a Spring Boot application, controllers can define endpoints that receive and process supported HTTP requests. The exact method and endpoint depend on the functionality being implemented.

For ChatSphere, HTTP-based communication provides a foundation for loading the application and supporting backend operations such as registration. However, ordinary HTTP request-response communication alone does not provide continuous, bidirectional real-time messaging.

4.7 User Registration

User registration is a common feature of web applications that allows individuals to provide the information required to create an account. A registration process typically includes a form where users enter the requested details and submit them for processing.

A registration system generally involves three stages:

1. **Input Collection:** The interface collects the information entered by the user.
2. **Validation and Processing:** The backend checks the submitted information and processes the request according to the application's rules.
3. **Response:** The application informs the user whether the operation was successful or requires correction.

Input validation helps prevent incomplete or incorrectly formatted information from being processed. Validation can take place in the browser for usability, but important checks must also be performed on the server because client-side checks can be bypassed.

Depending on the application's design, a registration system may also require persistent storage, password hashing, duplicate-account checks, and identity verification. These functions must be implemented and tested before they can be considered part of a working system.

ChatSphere includes a registration process. This feature provides an entry point for users to access the application and demonstrates the integration of a web form with backend functionality.

4.8 Theory of Real-Time Chat Applications

A chat application enables users to exchange messages through a digital interface. Unlike traditional communication methods, online messaging can deliver information across different locations without requiring the participants to be physically present together.

A messaging application generally contains a user interface, a communication mechanism, message-handling logic, and some method of identifying the users involved in a conversation. More advanced systems may also require message storage, delivery status, notifications, and access controls.

Real-time communication aims to deliver updates with minimal delay. The technologies used depend on the application's requirements.

Two common approaches are:

Polling: The client periodically sends requests to check whether new information is available. This approach is comparatively straightforward but can create unnecessary requests and may introduce a delay between updates.

WebSocket: WebSocket supports persistent, bidirectional communication between a client and server over a connection. Once established, either side can send messages without initiating a new HTTP request for every message. This makes WebSocket suitable for applications that require frequent, low-latency updates.

ChatSphere is intended to provide a web-based messaging experience. The technologies and communication mechanisms actually implemented determine whether messages are delivered in real time. Features such as persistent message storage, delivery acknowledgements, and group messaging require additional implementation where they are not already present.

4.9 Security in Web Applications

Security is an important consideration in applications that allow users to register or communicate online. A secure application should protect user information, validate incoming data, and prevent unauthorized access to protected functionality.

Common security practices include:

1. **Input Validation:** Checks that submitted information follows the expected format and rules.
2. **Password Protection:** Passwords should be stored using an appropriate password-hashing mechanism rather than as plain text.
3. **Authentication:** Verifies the identity of a user before granting access to account-specific functionality.
4. **Authorization:** Determines which actions an authenticated user is permitted to perform.
5. **Secure Communication:** HTTPS helps protect information transferred between the browser and server.
6. **Error Management:** Error responses should not reveal unnecessary sensitive implementation details.

These practices represent important security considerations for ChatSphere and similar applications. Their inclusion in the final implementation should be confirmed through the actual code and tests rather than assumed from the use of Spring Boot alone.

4.10 Error Handling and Input Validation

Input validation and error handling improve the reliability and usability of web applications. Input validation determines whether submitted data satisfies the application's requirements, while error handling manages unexpected situations during processing.

For example, a registration form may need to check whether required fields are completed and whether an email address follows an accepted format. If the submitted information is invalid, the application should provide a suitable response that helps the user correct the problem.

On the backend, exception handling can prevent unexpected errors from terminating an entire operation without a controlled response. Clear error messages and appropriate HTTP status codes also help distinguish successful requests from invalid input or server-side failures.

In ChatSphere, validation and error handling are relevant to registration and other supported interactions. Appropriate implementation helps produce more predictable behaviour and makes the application easier to test and maintain.

4.11 Maintainability and Future Enhancement

Maintainability refers to the ease with which an application can be corrected, modified, tested, and extended. A maintainable application generally uses clear naming, organized code, separation of responsibilities, and consistent coding practices.

The use of Java and Spring Boot provides opportunities to structure backend operations into separate components. Likewise, keeping HTML, CSS, and JavaScript organized makes the frontend easier to manage.

ChatSphere can be extended according to future requirements. Potential enhancements include group conversations, persistent message history, online-status indicators, notifications, file sharing, and stronger account security. Such enhancements would require appropriate design, implementation, and testing.

A modular structure can make these improvements easier to introduce without unnecessarily affecting existing functionality. Regular testing is also important to ensure that new features do not break previously working parts of the application.

In conclusion, the technologies and concepts discussed in this chapter provide the theoretical foundation for ChatSphere. The integration of frontend technologies, Java, Spring Boot, and client-server communication demonstrates the principles involved in developing a web-based application. These concepts also provide a basis for improving the application's functionality and reliability in future versions.

CHAPTER 5

PROCEDURE

The development of ChatSphere involves project setup, backend configuration, frontend development, user registration, integration, and testing. The application is developed using Java, Spring Boot, HTML, CSS, and JavaScript.

5.1 Project Setup and Configuration

The first step is to prepare the development environment. Visual Studio Code is used to manage the project files, Java is used for backend development, and Maven manages the project dependencies.

The following steps are performed:

1. Create the project folder named `ChatSphere`.
2. Configure the Spring Boot Maven project.
3. Verify the required Java and Maven settings.
4. Organize the source code, static resources, and configuration files.
5. Prepare the application for execution.

5.2 Backend Development

The backend is developed using Java and Spring Boot. It handles server-side operations and processes requests received from the frontend.

The procedure includes creating the main application class, configuring the required dependencies, implementing the necessary backend components, and setting the server port. Spring Boot provides an embedded web server that allows the application to run locally.

5.3 Frontend Development

The frontend provides the interface through which users interact with ChatSphere. It is developed using the following technologies:

- **HTML:** Defines the structure of webpages, forms, buttons, and chat elements.
- **CSS:** Provides styling, colours, layouts, spacing, and visual consistency.
- **JavaScript:** Handles user interactions, form submission, and communication with backend services.

These technologies are combined to create a user-friendly interface for registration and chat-related operations.

5.4 User Registration and Chat Interface

The registration module allows users to enter the required information through a registration form. The submitted information is processed according to the application's backend logic, and the interface displays an appropriate response.

The chat interface is designed to provide an area for viewing messages and entering new message content. HTML defines the interface elements, CSS styles the layout, and JavaScript handles user interactions.

The application's messaging mechanism must connect the chat interface to the backend to support communication between different users.

5.5 Frontend and Backend Integration

Integration connects the frontend with the Spring Boot backend. When a user performs an action, the frontend sends the relevant request to the server. The backend processes the request and returns a response, which the frontend uses to update the interface.

The integration process includes configuring endpoints, handling requests and responses, checking errors, and testing communication between the client and server.

Proper integration ensures that the different application components work together as intended.

5.6 Execution and Testing

After completing the configuration and integration, the application is executed and tested.The following steps are performed:

1. Open the ChatSphere project in Visual Studio Code.
2. Open the terminal in the project directory.
3. Run the application using the Maven Wrapper:

```
.\mvnw.cmd spring-boot:run
```

4. Wait for the Spring Boot server to start successfully.
5. Open the configured local address in a web browser.
6. Check the registration form and chat interface.
7. Test message exchange between two browser windows or user sessions.
8. Record the actual results and capture screenshots for the report.

Testing helps identify problems in registration, interface interactions, backend communication, and message delivery. Each feature should be marked successful only after its expected behaviour has been verified.

S. No.	Test Case	Expected / Checked Result
1	Application startup	Application opens in the browser
2	User registration	Registration process completes successfully
3	Dashboard display	Main dashboard and chat rooms are displayed
4	Chat interface	Message area and input field are visible

5	Message exchange	Messages are displayed in the two user windows
6	Message options	Reply, Edit, and Delete options are displayed

Table 3: Functional Testing of ChatSphere

In conclusion, the procedure follows a systematic approach to developing ChatSphere, from initial project configuration to application execution and functional testing. It demonstrates the practical application of full-stack development concepts in building a web-based communication system.

CHAPTER 6

CODE

6.1 Main Application Class

File: `ChatSphereApplication.java`

```
@SpringBootApplication
public class ChatSphereApplication {
    public static void main(String[] args) {
        SpringApplication.run(
            ChatSphereApplication.class, args);
    }}
```

6.2 Application Configuration

File: `application.properties`

```
spring.application.name=chatsphere
server.port=8081
```

6.3 User Model

File: `User.java`

```
public class User {
    private String username;
    private String email;
    private String password;}
```

6.4 Registration Controller

File: `RegistrationController.java`

```
@RestController
@RequestMapping("/api")
public class RegistrationController {
   @PostMapping("/register")
   public ResponseEntity<?> register(
         @RequestBody User user) {
      // Process user registration
   }}
```

6.5 Chat Message Model

File: `ChatMessage.java`

```
public class ChatMessage {
   private String sender;
   private String content;}
```

6.6 WebSocket Configuration

File: `WebSocketConfig.java`

```
@Configuration
@EnableWebSocket
public class WebSocketConfig
      implements WebSocketConfigurer {
   @Override
```

```
    public void registerWebSocketHandlers(
            WebSocketHandlerRegistry registry) {
        // Register the chat handler
    }}
```

6.7 Chat WebSocket Handler

File: `ChatWebSocketHandler.java`

```
public class ChatWebSocketHandler
        extends TextWebSocketHandler {
    @Override
    protected void handleTextMessage(
            WebSocketSession session,
            TextMessage message) {
        // Process incoming chat messages
    }}
```

6.8 Main Webpage

File: `index.html`

```
<!DOCTYPE html>
<html>
<head>
    <title>ChatSphere</title>
    <link rel="stylesheet" href="style.css">
</head>
```

```
<body>
    <h1>ChatSphere</h1>
    <div id="messages"></div>
    <input id="messageInput" placeholder="Type a message">
    <button id="sendButton">Send</button>
    <script src="script.js"></script>
</body>
</html>
```

6.9 CSS Styling

File: `style.css`

```
body {
    font-family: Arial, sans-serif;
    margin: 20px;}
#messages {
    min-height: 200px;
    padding: 10px;}
button {
    cursor: pointer;}
```

6.10 Frontend JavaScript

File: `script.js`

```
const socket = new WebSocket(
    "ws://localhost:8081/chat"
```

```
);

socket.onmessage = function(event) {

    // Display the received message

};

function sendMessage(message) {

    if (socket.readyState === WebSocket.OPEN) {

        socket.send(message);

    }}
```

CHAPTER 7

RESULTS

7.1 Sign-In Interface

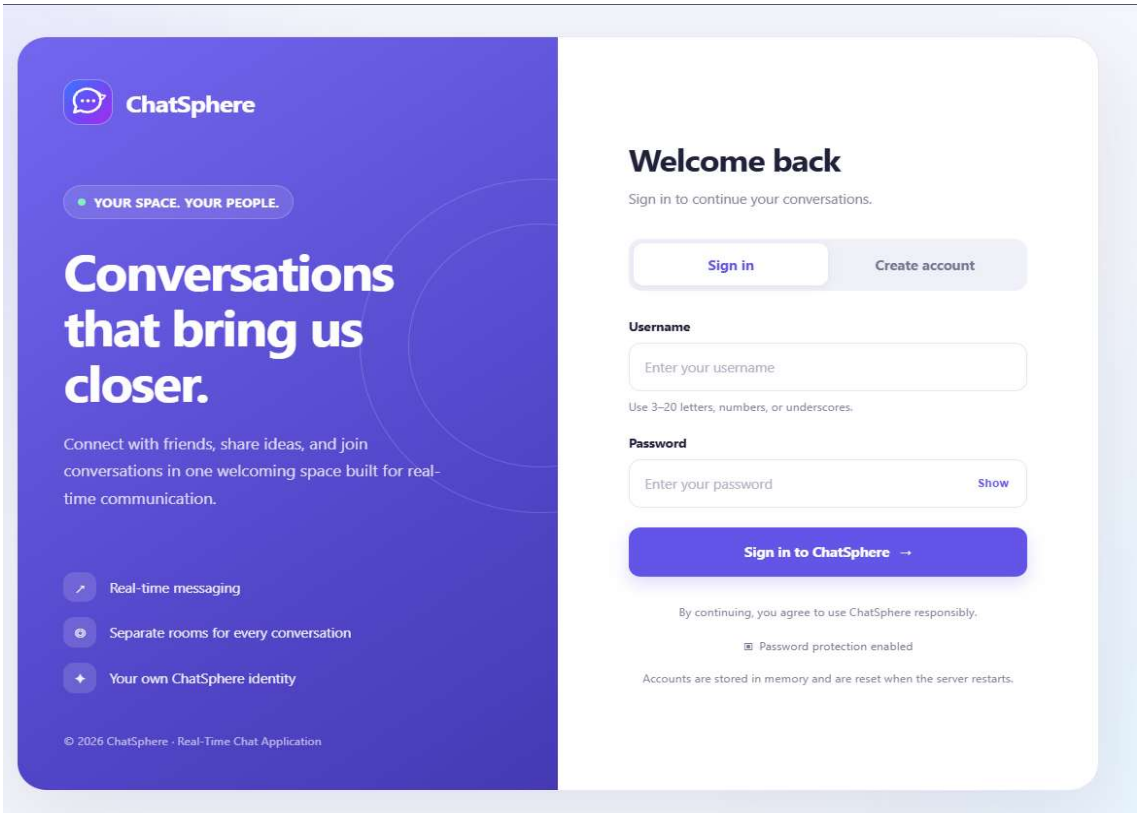


Fig. 7.1: ChatSphere Sign-In Interface

The ChatSphere sign-in page provides fields for entering a username and password. It also includes a Create Account option and introduces the application's main communication features. The interface uses a purple-and-white design with clear input fields and navigation controls.

7.2 Main Chat Dashboard



Fig. 7.2: ChatSphere Main Chat Dashboard

The main dashboard displays the user's profile, connection status, and available chat rooms, including General, Hangout, and Ideas. It also contains a message input field and a send button. The layout provides convenient access to the different conversation spaces.

7.3 Conversation from the First User's View

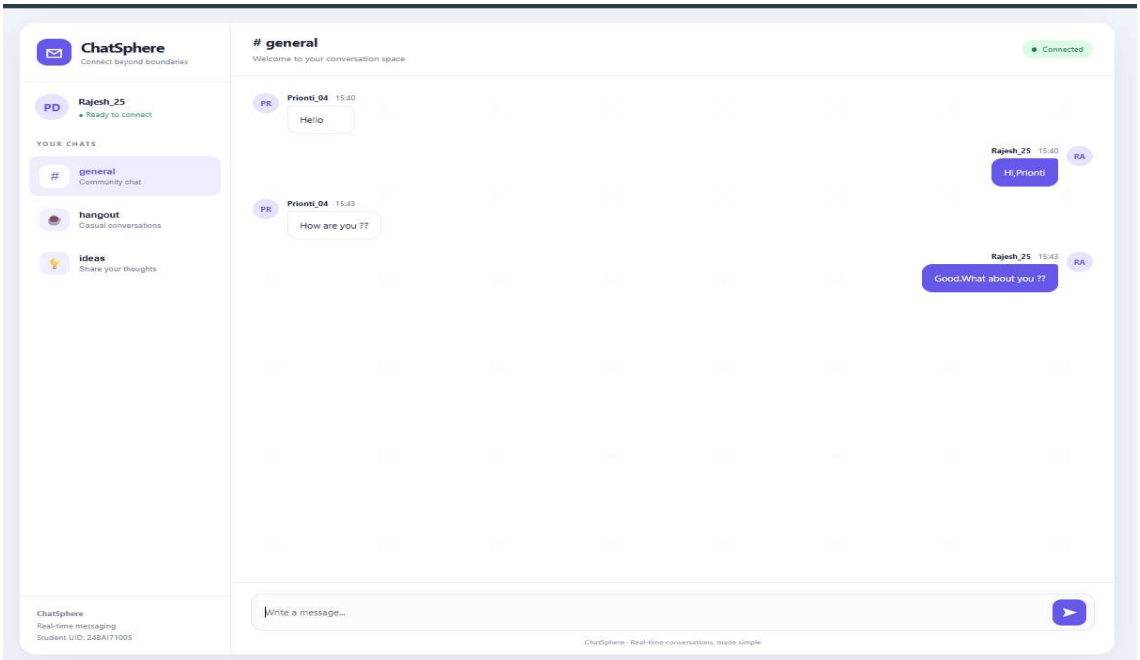


Fig. 7.3: Chat Conversation from the First User's View

The conversation interface displays messages exchanged between Prionti_04 and Rajesh_25. Messages such as “Hello,” “Hi, Prionti,” and “How are you?” are displayed with sender names and timestamps. The interface distinguishes between messages sent by the current user and those received from another user.

7.4 Conversation from the Second User's View

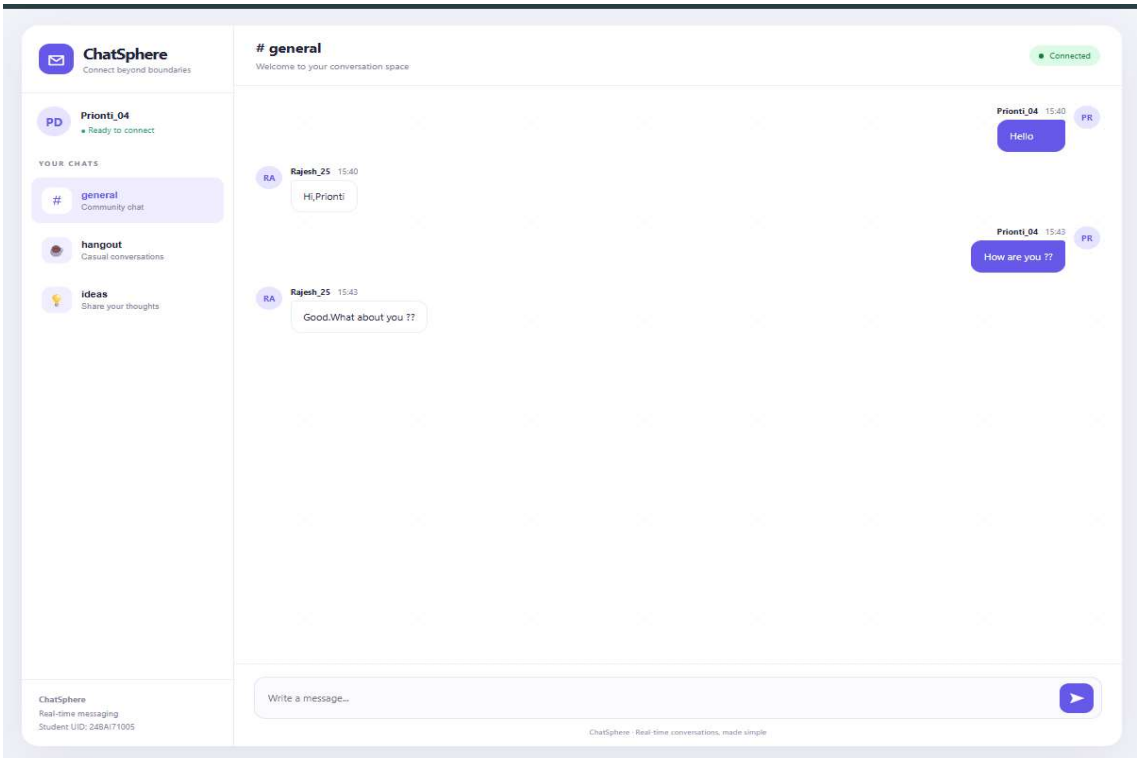


Fig. 7.4: Chat Conversation from the Second User's View

This screenshot shows the conversation from the other user's perspective. The same messages appear on opposite sides depending on the sender, demonstrating the interface's presentation of sent and received messages.

7.5 Ideas Chat Room

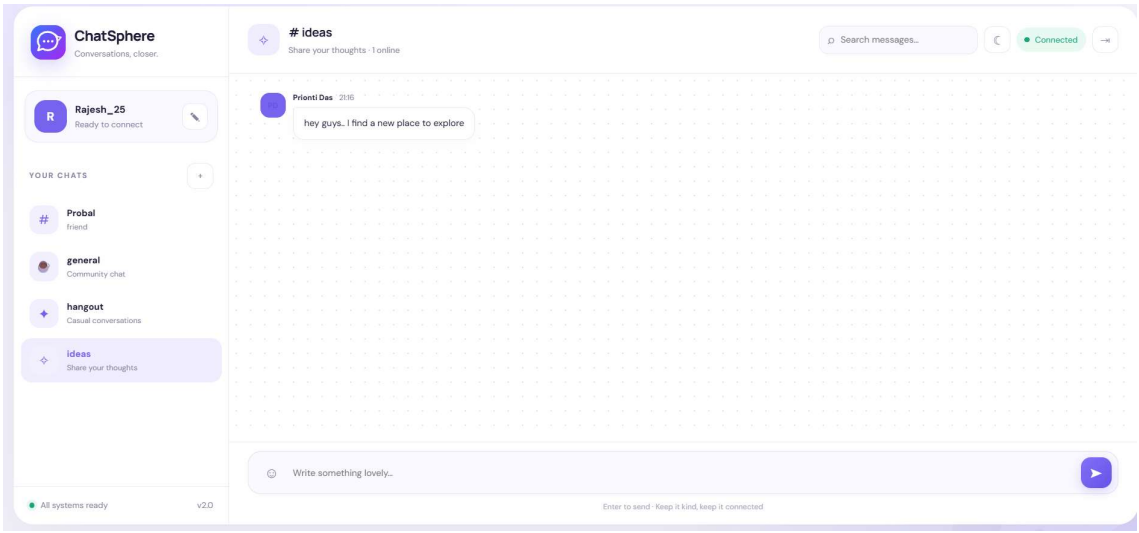


Fig. 7.5: Ideas Chat Room Interface

The Ideas chat room provides a dedicated space for sharing thoughts and discussing new ideas. The screenshot displays the selected room, a message sent by Prionti Das, the connection indicator, and the message input area. The search field is also available for finding messages.

7.6 Friend Conversation Interface

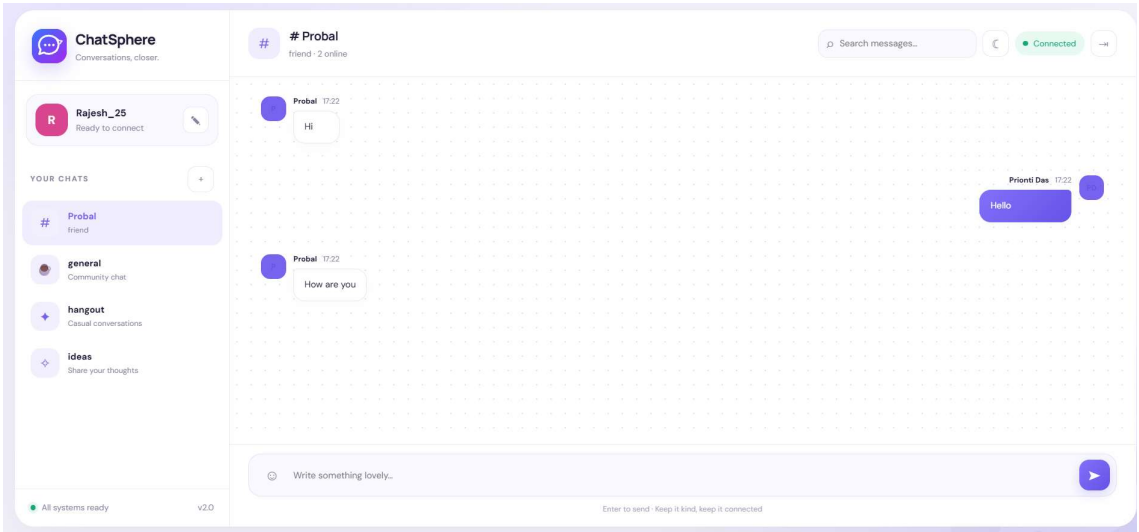


Fig. 7.6: Friend Conversation Interface

The Probal chat room demonstrates a conversation between users through a dedicated chat interface. The screenshot displays messages such as “Hi,” “Hello,” and “How are you?” along with sender names and timestamps. The selected conversation is highlighted in the sidebar.

7.7 General Community Chat

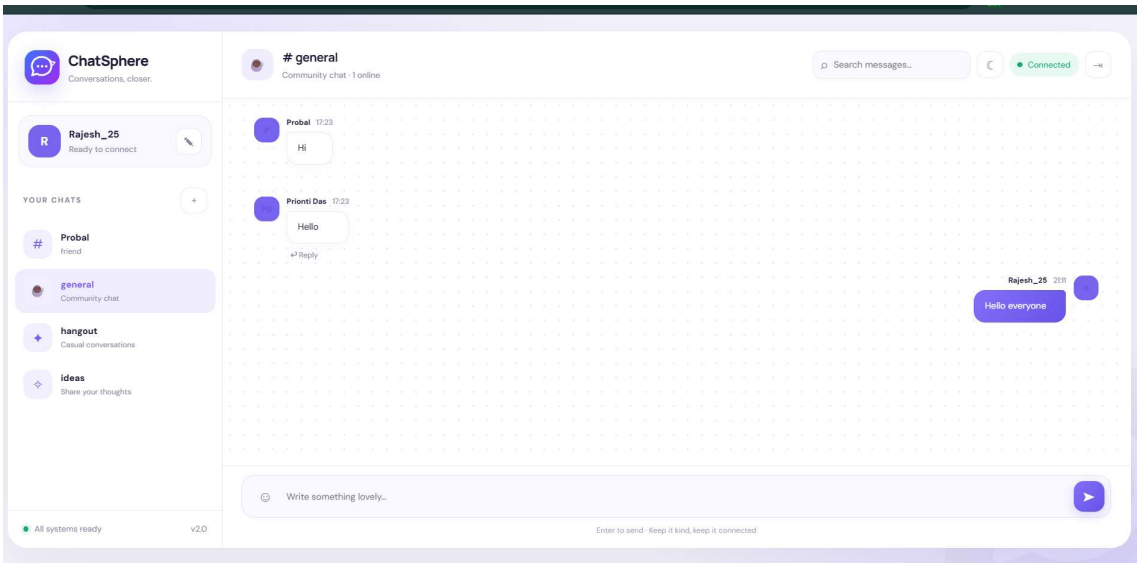


Fig. 7.7: General Community Chat Interface

The General chat room provides a shared space for community-style conversations. The screenshot displays messages from different users, the selected General room, and the message input field. The interface also includes navigation options for accessing other chat rooms.

7.8 Message Exchange Across Two Browser Windows

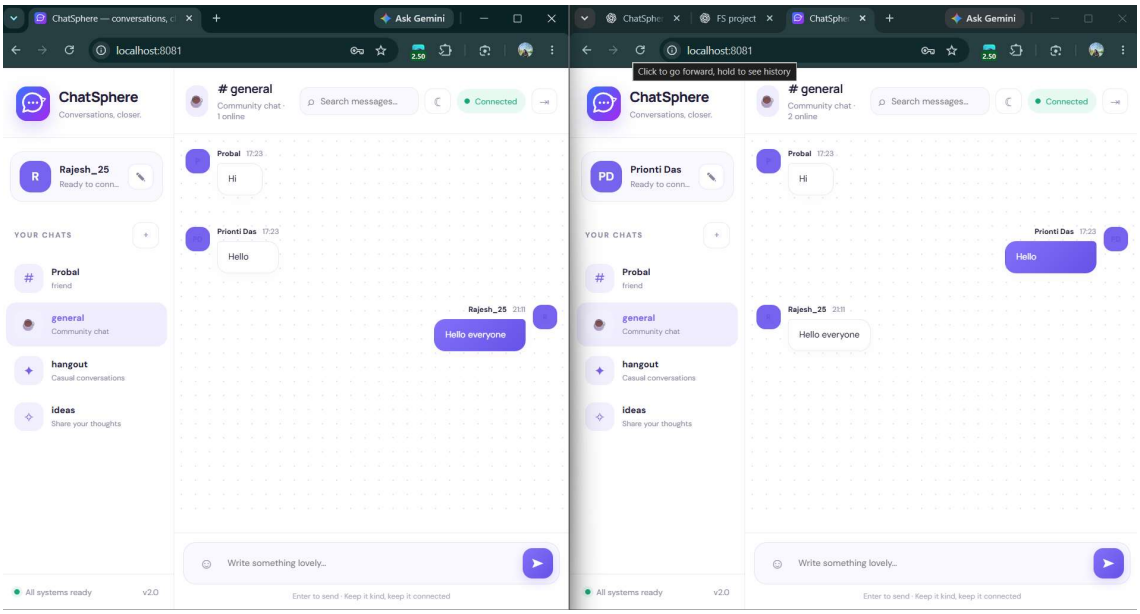


Fig. 7.8: Message Exchange Across Two Browser Windows

This screenshot shows ChatSphere open in two browser windows, with two different user sessions accessing the General chat room. Messages from both users are visible in the respective windows, providing visual evidence of message exchange across separate browser sessions.

7.9 Message Interaction Options

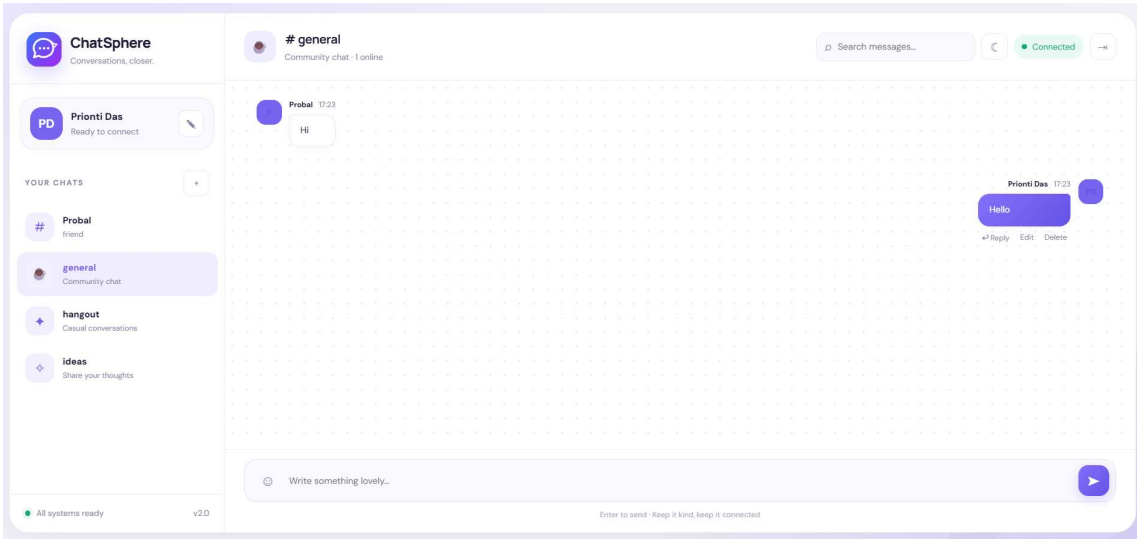


Fig. 7.9: Message Interaction Options

The chat interface displays additional options associated with a message, including Reply, Edit, and Delete. These options provide controls for interacting with individual messages and demonstrate the additional functionality available in the conversation interface.

S. No.	Feature	Observed Result
1	Sign-in interface	Interface displayed
2	Main dashboard	Dashboard and chat rooms displayed
3	Multiple chat rooms	General, Hangout, and Ideas interfaces displayed
4	User conversations	Messages shown with sender names and timestamps
5	Two-window messaging	Conversation displayed in both browser windows
6	Message interaction options	Reply, Edit, and Delete options displayed

Table 4: Summary of ChatSphere Features

7.10 Overall Results

The ChatSphere project demonstrates the development of a web-based communication application using Java, Spring Boot, HTML, CSS, and JavaScript. The screenshots present the sign-in interface, the main dashboard, separate chat rooms, and conversations between users.

The results demonstrate the following features:

1. The application provides a sign-in interface with an option to create an account.
2. The dashboard displays user information, connection status, and available chat rooms.
3. The General, Hangout, Ideas, and friend conversation interfaces provide different spaces for communication.
4. Messages are displayed with sender names and timestamps.
5. The two-window screenshot demonstrates message exchange between separate browser sessions.
6. The message interface displays Reply, Edit, and Delete options.

Overall, ChatSphere provides an interactive interface for web-based communication and demonstrates the integration of frontend and backend technologies in full-stack application development. The screenshots document the implemented interface and observed conversation workflow.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

ChatSphere is a web-based Real-Time Chat Application developed to provide a simple and interactive platform for online communication. The project demonstrates the use of Java and Spring Boot for backend development, along with HTML, CSS, and JavaScript for creating the user interface.

The application includes a sign-in interface, a registration option, a main dashboard and separate conversation spaces such as General, Hangout, Ideas and friend chats. The interface displays messages with sender names and timestamps and provides message interaction options, including Reply, Edit and Delete. The screenshots also demonstrate a sample conversation displayed across two browser windows.

The development of ChatSphere provided practical knowledge of full-stack web development, frontend and backend integration, client-server communication, and user interface design. It also helped in understanding the importance of application testing, message handling, and user experience.

Overall, ChatSphere establishes a foundation for a web-based messaging application that can be improved with additional functionality, stronger security, and more advanced communication features.

8.2 Future Scope

ChatSphere can be enhanced in several ways to improve its functionality, security, and usability.

1. **Persistent Message Storage:** A database can be integrated to retain user accounts and conversation history even after the server restarts.
2. **Group Chat Enhancement:** Additional features can be developed to support group creation, participant management, and group communication.

3. **Improved Security:** Stronger authentication, secure password management, and access controls can help protect user accounts and conversations.
4. **Notifications:** Real-time notifications can inform users when new messages arrive.
5. **File Sharing:** Users could be allowed to exchange documents, images, and other supported files through the chat interface.
6. **User Status:** Online and offline indicators can help users understand the availability of other participants.
7. **Responsive Design:** Further improvements can make the application easier to use on mobile phones, tablets, and different screen sizes.
8. **Message Management:** Message editing, deletion, reply, and search functionality can be refined to provide better conversation management.

These enhancements could make ChatSphere a more complete and reliable communication platform. They represent possible future developments and would require implementation and testing before being considered completed features.

Project Links

- GitHub Repository: https://github.com/Priontipayel05/ChatSphere
- Live Website (Frontend): ChatSphere | Welcome